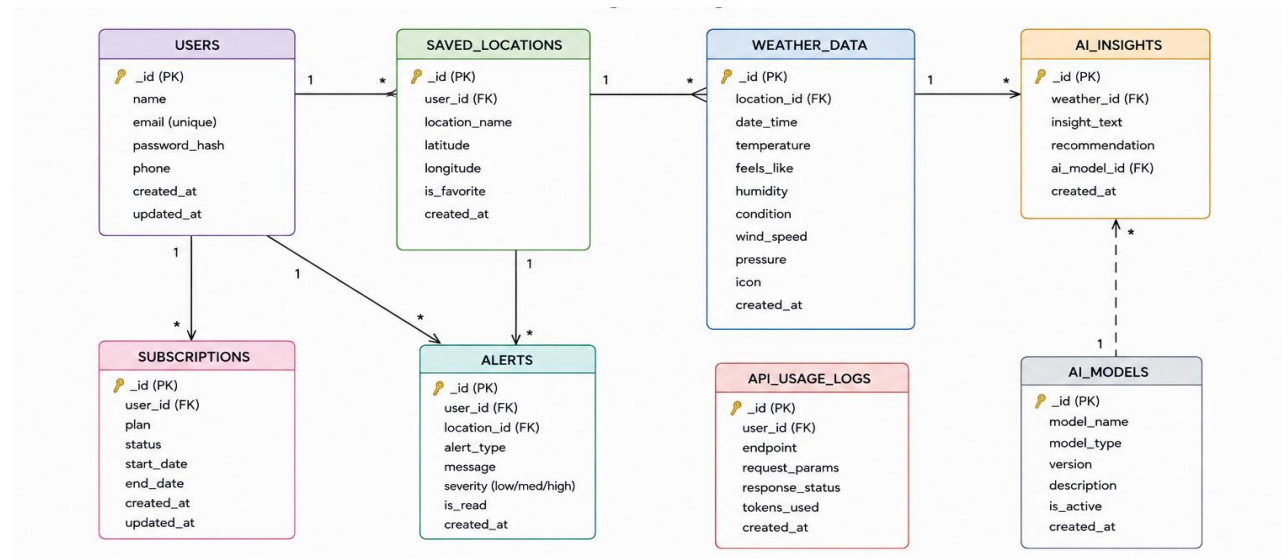


Phase 4

Project Planning Phase

Phase 4: Project Planning Phase



The data architecture manages User profiles and their associated favorite Locations through logical references:

1. User Entity

- _id (Primary Key - ObjectId)
- name (String, Required)
- email (String, Required, Unique)
- password (String, Hashed, Required)
- role (String, Default: "reader")

2. Location Entity

- _id (Primary Key - ObjectId)
- user (Foreign Key referencing User Collection)
- city (String, Required)
- country (String, Required)
- createdAt (Date, Default: Now)

Key Structural Mappings

- **User to Location (One-to-Many):** A registered user can save multiple favorite locations.

Key Features

- **Role-Based Route Guarding:** Strict token processing blocks unauthorized tiers from modifying core settings.
- **CRUD Blog Execution Engine:** Complete schema management over content definitions, status states, and asset assignment paths.
- **Categorization System:** Native tag indices facilitate filtering and querying for enhanced discoverability.
- **Comment Moderation Logic:** Protected entry points for updating moderation flags (approved, pending, spam).
- **Sanitization Safeguards:** Payload parameter sanitation prevents XSS vectors and NoSQL parameter manipulation injections.

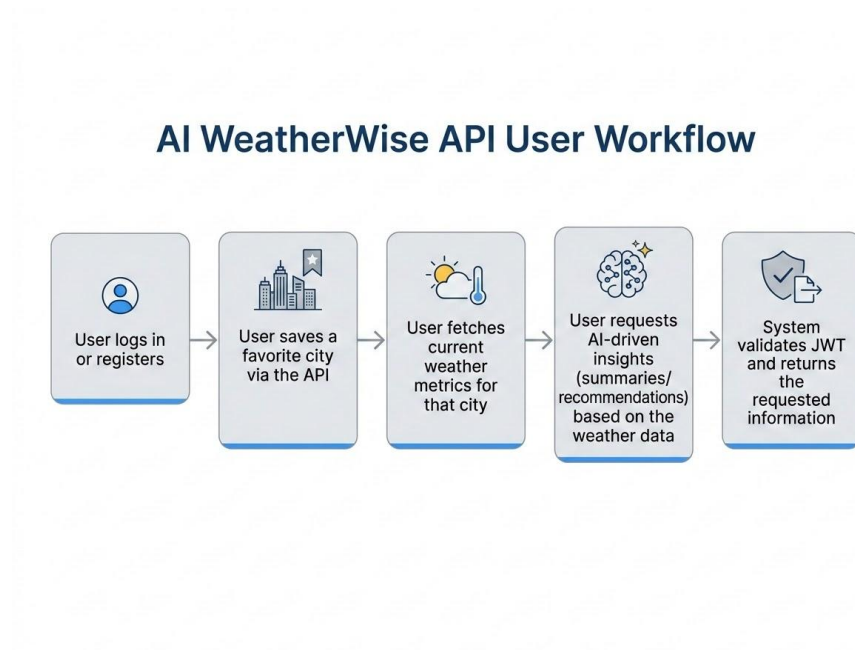
Roles and Responsibilities

- **Admin:**
 - Responsible for user account management, including creating, deleting, or suspending accounts.
 - Oversight of API health and system logs to monitor performance.
 - Monitoring and managing database integrity.
- **Registered User:**
 - Responsible for creating and maintaining a list of favorite locations.
 - Initiating requests for real-time weather data.
 - Interacting with the Gemini AI service for personalized weather summaries and activity recommendations.
 - Managing their own authentication credentials, such as updating profile details.

User Flow:

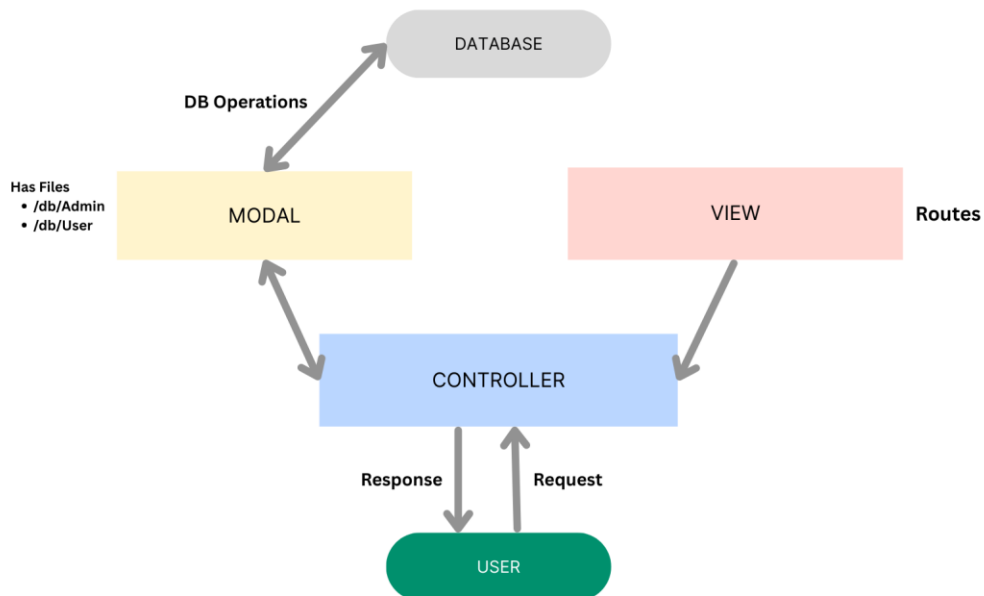
1. User logs in or registers.
2. User saves a favorite city via the API.
3. User fetches current weather metrics for that city.
4. User requests AI-driven insights (summaries/recommendations) based on the weather data.

5. System validates JWT and returns the requested information.



MVC Architecture Pattern

The system architecture structures its source configuration layers around the Model-View-Controller design pattern to completely decouple components:



Model Layer (Data Blueprint)

Defines structure blueprints via strongly enforced Mongoose schemas mapping explicit fields into raw collection items inside MongoDB. It sets default states, indexing rules, and transformation parameters before validation persistence occurs.

Controller Layer (Execution Matrix)

The intermediary brain of the ecosystem. It captures request vectors from the execution routes, checks structural request constraints, delegates processing workflows directly to models, and packages the resultant output into standard JSON return arrays.

View Layer (Routing Network Interface)

In this pure headless backend service context, standard user-interface template views do not exist. Instead, the interface layer lives as an API **Routing Layer**. It links network requests on specified paths directly to their associated controller handler logic maps.